```
import ssl

ssl._create_default_https_context = ssl._create_unverified_context

!pip -q install nltk tqdm

import os
import json
import zipfile
import random
import urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import nltk

from PIL import Image
from collections import Counter
from tqdm.auto import tqdm
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunct

from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import Input, Dense, Embedding, LSTM,
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping


SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

BASE_DIR = "/content/coco_captioning"
IMAGE_DIR = f"{BASE_DIR}/val2017"
ANNOTATION_DIR = f"{BASE_DIR}/annotations"

MAX_IMAGES = 1000
MAX_VOCAB = 8000
MAX_LEN = 30
EMBED_DIM = 256
LSTM_UNITS = 256
BATCH_SIZE = 256
EPOCHS = 3
```

```
EPOCHS = 5

os.makedirs(IMAGE_DIR, exist_ok=True)
os.makedirs(ANNOTATION_DIR, exist_ok=True)

print("TensorFlow:", tf.__version__)
print("Available GPUs:", tf.config.list_physical_devices("GPU"))



def download_file(url, path):
    if not os.path.exists(path):
        print("Downloading:", os.path.basename(path))

        context = ssl._create_unverified_context()
        with urllib.request.urlopen(url, context=context) as respo
            data = response.read()
            out_file.write(data)
    else:
        print("Already downloaded:", os.path.basename(path))

download_file(
    "https://images.cocodataset.org/zips/val2017.zip",
    f"{BASE_DIR}/val2017.zip"
)

download_file(
    "https://images.cocodataset.org/annotations/annotations_trainva
    f"{BASE_DIR}/annotations.zip"
)

if not os.path.exists(f"{IMAGE_DIR}/000000000139.jpg"):
    with zipfile.ZipFile(f"{BASE_DIR}/val2017.zip") as z:
        z.extractall(BASE_DIR)

if not os.path.exists(f"{ANNOTATION_DIR}/captions_val2017.json"):
    with zipfile.ZipFile(f"{BASE_DIR}/annotations.zip") as z:
        z.extractall(BASE_DIR)

print("COCO dataset is ready.")



with open(f"{ANNOTATION_DIR}/captions_val2017.json", "r") as f:
    coco = json.load(f)

image_lookup = {
    item["id"]: item["file_name"]
    for item in coco["images"]
```

```
}

captions_by_image = {}

for item in coco["annotations"]:
    img_id = item["image_id"]
    caption = item["caption"].lower().strip()
    captions_by_image.setdefault(img_id, []).append(caption)

valid_ids = [
    img_id for img_id in image_lookup
    if img_id in captions_by_image
    and os.path.exists(f"{IMAGE_DIR}/{image_lookup[img_id]}")
]

random.shuffle(valid_ids)
selected_ids = valid_ids[:MAX_IMAGES]

records = []

for img_id in selected_ids:
    for caption in captions_by_image[img_id]:
        records.append({
            "image_id": img_id,
            "filename": image_lookup[img_id],
            "caption": "startseq " + caption + " endseq"
        })

df = pd.DataFrame(records)

print("Selected images:", len(selected_ids))
print("Caption pairs:", len(df))
display(df.head())


tokenizer = Tokenizer(
    num_words=MAX_VOCAB,
    oov_token="<unk>",
    filters='!"#$%&()*+,-./:;=?@[\\]^_`{|}~\t\n'
)

tokenizer.fit_on_texts(df["caption"].tolist())

vocab_size = min(MAX_VOCAB, len(tokenizer.word_index) + 1)

df["sequence"] = tokenizer.texts_to_sequences(df["caption"])

df = df[
    df["sequence"].apply(lambda x: 2 <= len(x) <= MAX_LEN)
```

```
].reset_index(drop=True)

word_index = tokenizer.word_index
index_word = {v: k for k, v in word_index.items()}

print("Vocabulary size:", vocab_size)
print("Usable captions:", len(df))



cnn = ResNet50(
    weights="imagenet",
    include_top=False,
    pooling="avg",
    input_shape=(224, 224, 3)
)

cnn.trainable = False

print("CNN output shape:", cnn.output_shape)

def load_image(path):
    img = Image.open(path).convert("RGB").resize((224, 224))
    arr = np.asarray(img, dtype=np.float32)
    return preprocess_input(arr)

unique_ids = df["image_id"].unique()
image_features = {}

CNN_BATCH = 32

for start in tqdm(range(0, len(unique_ids), CNN_BATCH),
                  desc="Extracting CNN features"):
    batch_ids = unique_ids[start:start + CNN_BATCH]

    batch_images = np.stack([
        load_image(
            f"{IMAGE_DIR}/{image_lookup[int(img_id)]}"
        )
        for img_id in batch_ids
    ])

    features = cnn.predict(batch_images, verbose=0)

    for img_id, feature in zip(batch_ids, features):
        image_features[int(img_id)] = feature

print("Extracted image features:", len(image_features))
```

```
image_input = Input(shape=(2048,), name="image_features")

image_embedding = Dense(
    EMBED_DIM, activation="relu"
)(image_input)

image_embedding = Dropout(0.3)(image_embedding)

sequence_input = Input(
    shape=(MAX_LEN - 1,),
    name="caption_sequence"
)

text_embedding = Embedding(
    input_dim=vocab_size,
    output_dim=EMBED_DIM,
    mask_zero=True,
    name="word_embedding"
)(sequence_input)

lstm1 = LSTM(
    LSTM_UNITS,
    return_sequences=True,
    name="lstm_1"
)(text_embedding)

lstm1 = Dropout(0.3)(lstm1)

lstm2 = LSTM(
    LSTM_UNITS,
    name="lstm_2"
)(lstm1)

text_projection = Dense(
    EMBED_DIM, activation="relu"
)(lstm2)

combined = Add()([
    image_embedding,
    text_projection
])

combined = Dense(256, activation="relu")(combined)
combined = Dropout(0.3)(combined)

output = Dense(
```

```
    vocab_size,
    activation="softmax",
    name="next_word"
)(combined)

model = Model(
    inputs=[image_input, sequence_input],
    outputs=output
)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["sparse_categorical_accuracy"]
)

model.summary()


X_image = []
X_sequence = []
y_next_word = []

for row in tqdm(df.itertuples(index=False),
                total=len(df),
                desc="Preparing sequences"):

    img_id = int(row.image_id)
    seq = row.sequence

    if img_id not in image_features:
        continue

    for i in range(1, len(seq)):
        target = seq[i]

        if target >= vocab_size:
            continue

        input_seq = seq[:i]

        padded_seq = pad_sequences(
            [input_seq],
            maxlen=MAX_LEN - 1,
            padding="post",
            truncating="post"
        )[0]

        X_image.append(image_features[img_id])
```

```
        X_sequence.append(padded_seq)
        y_next_word.append(target)

X_image = np.asarray(X_image, dtype=np.float32)
X_sequence = np.asarray(X_sequence, dtype=np.int32)
y_next_word = np.asarray(y_next_word, dtype=np.int32)

print("Image input:", X_image.shape)
print("Sequence input:", X_sequence.shape)
print("Target output:", y_next_word.shape)


history = model.fit(
    [X_image, X_sequence],
    y_next_word,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS,
    validation_split=0.1,
    shuffle=True,
    callbacks=[
        EarlyStopping(
            monitor="val_loss",
            patience=2,
            restore_best_weights=True
        )
    ],
    verbose=1
)

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].plot(history.history["loss"], marker="o",
             label="Training loss")
axes[0].plot(history.history["val_loss"], marker="o",
             label="Validation loss")
axes[0].set_title("Model Loss")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Cross-entropy loss")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(
    history.history["sparse_categorical_accuracy"],
    marker="o",
    label="Training accuracy"
)
axes[1].plot(
    history.history["val_sparse_categorical_accuracy"],
```

```
    marker="o",
    label="Validation accuracy"
)
axes[1].set_title("Next-Word Prediction Accuracy")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()


def generate_caption(img_id):
    feature = image_features[int(img_id)]
    caption = "startseq"

    for _ in range(MAX_LEN - 1):
        seq = tokenizer.texts_to_sequences([caption])

        seq = pad_sequences(
            seq,
            maxlen=MAX_LEN - 1,
            padding="post",
            truncating="post"
        )

        probs = model.predict(
            [feature[np.newaxis, :], seq],
            verbose=0
        )[0]

        next_id = int(np.argmax(probs))
        next_word = index_word.get(next_id, "")

        if next_id == 0 or next_word in ("", "endseq", "startseq"):
            break

        caption += " " + next_word

    return caption.replace("startseq", "").strip()


def show_caption(img_id):
    img_id = int(img_id)
    filename = image_lookup[img_id]
    image_path = f"{IMAGE_DIR}/{filename}"

    img = Image.open(image_path).convert("RGB")
```

```
    generated = generate_caption(img_id)

    plt.figure(figsize=(9, 6))
    plt.imshow(img)
    plt.axis("off")
    plt.title(
        "GENERATED CAPTION\n" + generated,
        fontsize=13,
        wrap=True
    )
    plt.show()

    print("Generated:", generated)
    print("\nReference captions:")

    for cap in captions_by_image[img_id][:5]:
        print("-", cap)



test_id = random.choice(list(image_features.keys()))
show_caption(test_id)



smooth = SmoothingFunction().method1

evaluation_ids = list(image_features.keys())[:50]
bleu_scores = []

for img_id in tqdm(evaluation_ids, desc="Evaluating BLEU-1"):
    predicted = generate_caption(img_id).split()

    references = [
        caption.split()
        for caption in captions_by_image[int(img_id)]
    ]

    score = sentence_bleu(
        references,
        predicted,
        weights=(1, 0, 0, 0),
        smoothing_function=smooth
    )

    bleu_scores.append(score)

mean_bleu = float(np.mean(bleu_scores))
```

```
print("\n========== EVALUATION RESULTS ==========")
print("Images evaluated:", len(bleu_scores))
print(f"Mean BLEU-1: {mean_bleu:.4f}")

plt.figure(figsize=(8, 4))
plt.hist(bleu_scores, bins=10, edgecolor="black")
plt.title("Distribution of BLEU-1 Scores")
plt.xlabel("BLEU-1 score")
plt.ylabel("Number of images")
plt.grid(True, alpha=0.3)
plt.show()



model.save("/content/image_caption_model.keras")

with open("/content/caption_tokenizer.json", "w") as f:
    f.write(tokenizer.to_json())

np.savez_compressed(
    "/content/coco_image_features.npz",
    **{
        str(img_id): feature
        for img_id, feature in image_features.items()
    }
)

print("\n========== PROJECT COMPLETED ==========")
print("Saved model: /content/image_caption_model.keras")
print("Saved tokenizer: /content/caption_tokenizer.json")
print("Saved CNN features: /content/coco_image_features.npz")



from google.colab import files

files.download("/content/image_caption_model.keras")
files.download("/content/caption_tokenizer.json")
```

```
TensorFlow: 2.21.0
Available GPUs: []
Downloading: val2017.zip
Downloading: annotations.zip
COCO dataset is ready.
Selected images: 1000
Caption pairs: 5002
```

	image_id	filename	caption
0	21465	000000021465.jpg	startseq a dirty table has incense burners on ...
1	21465	000000021465.jpg	startseq there are various items on shelves ta...
2	21465	000000021465.jpg	startseq a cabinet with some paint on top of i